1. Let us name the algorithm square_root.
2. It takes the number as the input. Let us name the input n. n should not be negative.
3. It produces the square root of n as the output. Let us name the output y. Then n should be the square of y.

Now the specification of the algorithm is

square_root(n)

-- inputs: n is a real number, n ≥ 0.

-- outputs: y is a real number such that $y^2 = n$.

6.5.1 Specification as contract

Specification of an algorithm serves as a contract between the designer of the algorithm and the users of the algorithm, because it defines the rights and responsibilities of the designer and the user.

Ensuring that the inputs satisfy the required properties is the responsibility of the user, but the right of the designer. The desired input-output relation is the responsibility of the designer and the right of the user. Importantly, if the user fails to satisfy the properties of the inputs, the designer is free from his obligation to satisfy the desired input-output relation.

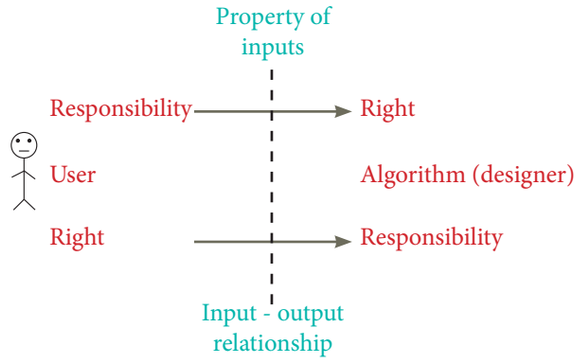


Figure 6.2: Input property and the input-output relation as rights and responsibilities

Example 6.8. Consider the specification of the algorithm square_root.

square_root(n)

-- inputs: n is a real number, n ≥ 0.

-- outputs : y is a real number such that $y^2 = n$.

The algorithm designer can assume that the given number is non-negative, and construct the algorithm. The user can expect the output to be the square root of the given number.

The output could be the negative square root of the given number. The specification did not commit that the output is the positive square root. If the user passes a negative number as the input, then the output need not be the square root of the number.

6.6 Abstraction

To ride a bicycle, it is sufficient to understand the functioning of the pedal, handlebar, brakes and bell. As a rider, we model a bicycle by these four features. A bicycle has a lot more details, which the rider can ignore. Those details are irrelevant for the purpose of riding a bicycle.

A problem can involve a lot of details. Several of these details are irrelevant for solving the problem. Only a few details are essential. Abstraction is the process of ignoring or hiding irrelevant details and modeling a problem only by its essential features. In our everyday life, we use abstractions unconsciously to handle complexity. Abstraction is the most effective mental tool used for managing complexity. If we do not abstract a problem adequately, we may deal with unnecessary details and complicate the solution.

Chapter 6 Page 076-087.indd 81 3/24/2020 9:14:09 AM